Untitled

March 3, 2026

[1]:
```
import os
os.chdir('/Users/franciscegoat/Zermelo')
import matplotlib.pyplot as plt
import numpy as np
import src.utils.metrics as mt
from src.models.coefficients import f_1, compute_f_exact, u_exact
from config.parameters import Params
from src.models.fd.solver import Solveur_Zermelo




params = Params(
    r = .5,
    R = np.sqrt(2),
    kappa = .1,
    vs = .5,
    a = .2,
    sig_x = .5,
    sig_y = .2,
    X_min = -2.0,
    X_max = 2.0,
    Y_min = -2.0,
    Y_max = 2.0
)
```

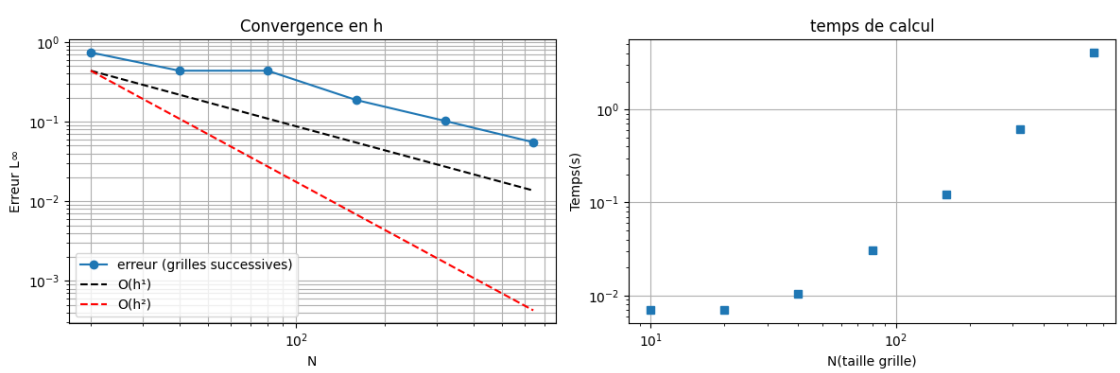
1 Notebook des résultats:

1.1 1. Différences finies :

[2]:
```
N_list = [10*2**(k) for k in range (7)]

errors, times, e = mt.convergence_fd(params, N_list, f_1)
```

[3]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

ax = axes[0]
```

```
ax.loglog(N_list[1:], errors, 'o-', label='erreur (grilles successives)')
ax.loglog(N_list[1:], [errors[1] * (N_list[1]/N_list[i+1])**1 for i in␣
 ↪range(len(errors))],
              'k--', label='O(h¹)')
ax.loglog(N_list[1:], [errors[1] * (N_list[1]/N_list[i+1])**2 for i in␣
 ↪range(len(errors))],
              'r--', label='O(h²)')
ax.set_xlabel('N'); ax.set_ylabel('Erreur L∞')
ax.set_title('Convergence en h')
ax.legend(); ax.grid(True, which='both')

ax2 = axes[1]
ax2.loglog(N_list, times, 's')
ax2.set_xlabel('N(taille grille)')
ax2.set_ylabel('Temps(s)')
ax2.set_title('temps de calcul')
#ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

print(times)
```

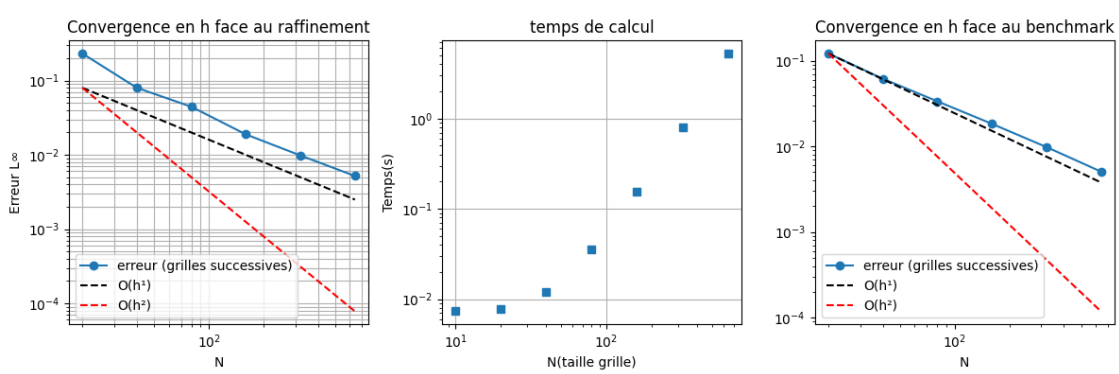



```
[0.006989002227783203, 0.006910085678100586, 0.01038217544555664,
0.030620098114013672, 0.12086296081542969, 0.6155221462249756,
4.091105699539185]
```

[4]:
```
errors, times, errex = mt.convergence_fd(params, N_list, compute_f_exact,␣
 ↪u_exacte = u_exact)
```

[5]:
```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
ax = axes[0]
ax.loglog(N_list[1:], errors, 'o-', label='erreur (grilles successives)')
```

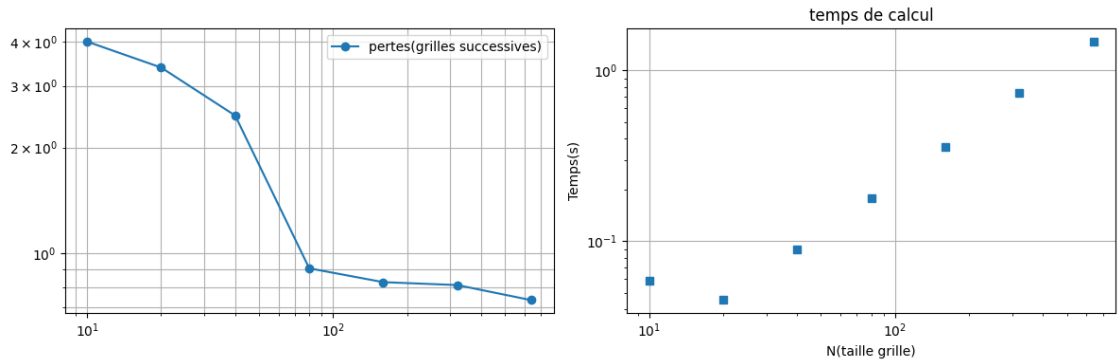
```
ax.loglog(N_list[1:], [errors[1] * (N_list[1]/N_list[i+1])**1 for i in␣
 ↪range(len(errors))],
              'k--', label='O(h¹)')
ax.loglog(N_list[1:], [errors[1] * (N_list[1]/N_list[i+1])**2 for i in␣
 ↪range(len(errors))],
              'r--', label='O(h²)')
ax.set_xlabel('N'); ax.set_ylabel('Erreur L∞')
ax.set_title('Convergence en h face au raffinement')
ax.legend(); ax.grid(True, which='both')

ax2 = axes[1]
ax2.loglog(N_list, times, 's')
ax2.set_xlabel('N(taille grille)')
ax2.set_ylabel('Temps(s)')
ax2.set_title('temps de calcul')
#ax2.legend()
ax2.grid(True)

ax3 = axes[2]
ax3.loglog(N_list[1:], errex[1:], 'o-', label='erreur (grilles successives)')
ax3.loglog(N_list[1:], [errex[1] * (N_list[1]/N_list[i+1])**1 for i in␣
 ↪range(len(errex)-1)],
              'k--', label='O(h¹)')
ax3.loglog(N_list[1:], [errex[1] * (N_list[1]/N_list[i+1])**2 for i in␣
 ↪range(len(errex)-1)],
              'r--', label='O(h²)')
ax3.set_xlabel('N'); ax.set_ylabel('Erreur L∞')
ax3.set_title('Convergence en h face au benchmark')
ax3.legend(); ax.grid(True, which='both')

plt.tight_layout()
plt.show()
print (times)
```



```
[0.007327079772949219, 0.007850885391235352, 0.011878013610839844,
0.03596210479736328, 0.15497326850891113, 0.79976487159729, 5.255701065063477]
```

[6]:
```
epochs = N_list
l_moy, times,e = mt.convergence_pinn_epochs(params, f_1, epoch_list = epochs)
```

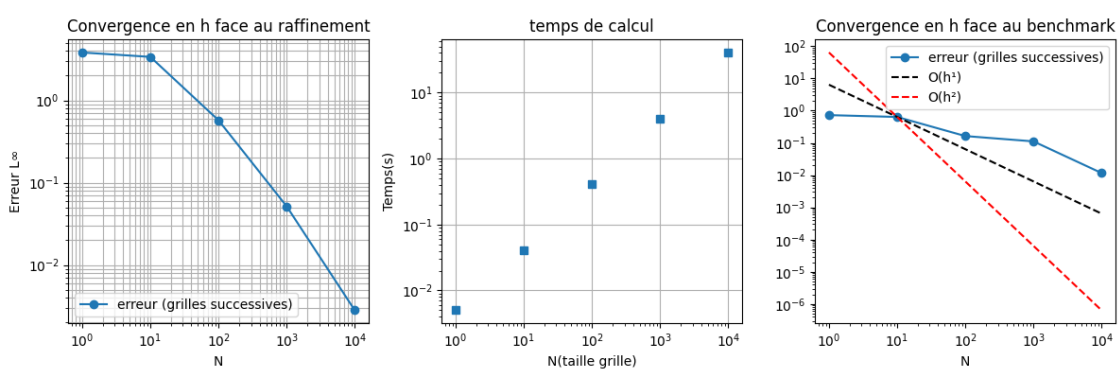
[7]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

ax = axes[0]
ax.loglog(N_list[:], l_moy, 'o-', label='pertes(grilles successives)')

ax.legend(); ax.grid(True, which='both')

ax2 = axes[1]
ax2.loglog(N_list, times, 's')
ax2.set_xlabel('N(taille grille)')
ax2.set_ylabel('Temps(s)')
ax2.set_title('temps de calcul')
#ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()
```



[8]:
```
epochs = [10**k for k in range (5)]
l_moy, times,e = mt.convergence_pinn_epochs(params, compute_f_exact, u_ref =␣
↪u_exact, epoch_list = epochs)
```

[9]:
```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
ax = axes[0]
ax.loglog(epochs[:], l_moy, 'o-', label='erreur (grilles successives)')
ax.set_xlabel('N'); ax.set_ylabel('Erreur Lω')
ax.set_title('Convergence en h face au raffinement')
ax.legend(); ax.grid(True, which='both')
```

```
ax2 = axes[1]
ax2.loglog(epochs[:], times, 's')
ax2.set_xlabel('N(taille grille)')
ax2.set_ylabel('Temps(s)')
ax2.set_title('temps de calcul')
#ax2.legend()
ax2.grid(True)
if e is not None :
    ax3 = axes[2]
    ax3.loglog(epochs[:], e[:], 'o-', label='erreur (grilles successives)')
    ax3.loglog(epochs[:], [e[1] * (epochs[1]/epochs[i])**1 for i in␣
 ↪range(len(errex)-2)],
               'k--', label='O(h¹)')
    ax3.loglog(epochs[:], [e[1] * (epochs[1]/epochs[i])**2 for i in␣
 ↪range(len(errex)-2)],
               'r--', label='O(h²)')
    ax3.set_xlabel('N'); ax.set_ylabel('Erreur Lω')
    ax3.set_title('Convergence en h face au benchmark')
    ax3.legend(); ax.grid(True, which='both')

    plt.tight_layout()
    plt.show()
```



[10]:
```
N = 100



import cProfile
##cProfile.run('Solveur_Zermelo(N, f_1, params)', sort='cumulative')
```

[11]:
```
help(cProfile.run)
```

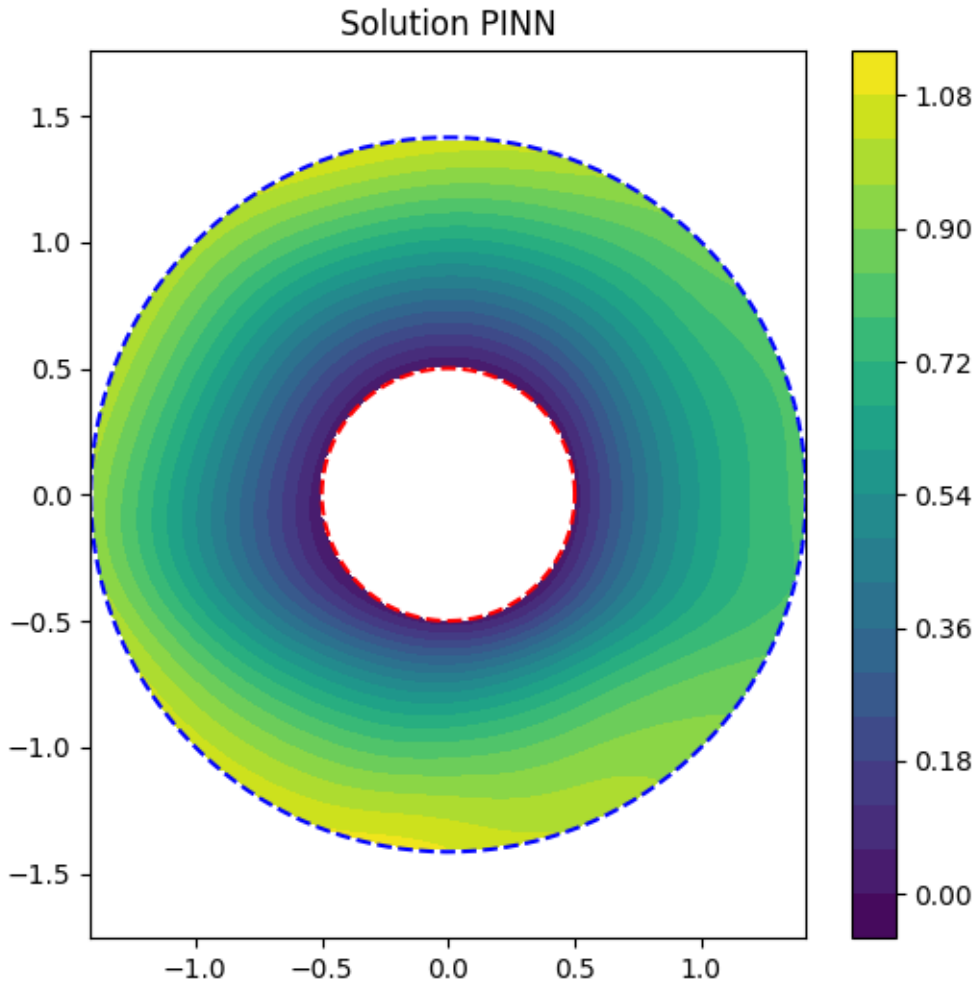
```
Help on function run in module cProfile:
```

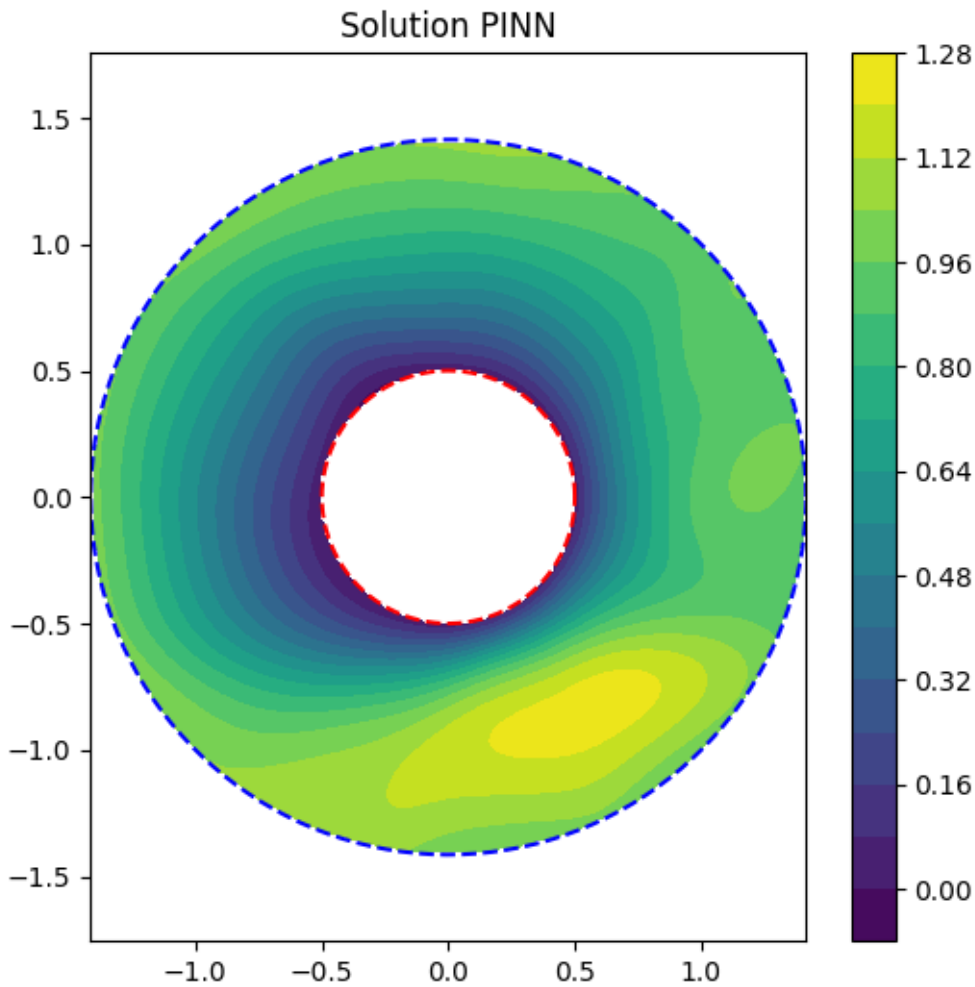
```
run(statement, filename=None, sort=-1)
    Run statement under profiler optionally saving results in filename

    This function takes a single argument that can be passed to the
    "exec" statement, and an optional file name.  In all cases this
    routine attempts to "exec" its first argument and gather profiling
    statistics from the execution. If no file name is present, then this
    function automatically prints a simple profiling report, sorted by the
    standard name string (file/line/function-name) that is presented in
    each line.
```

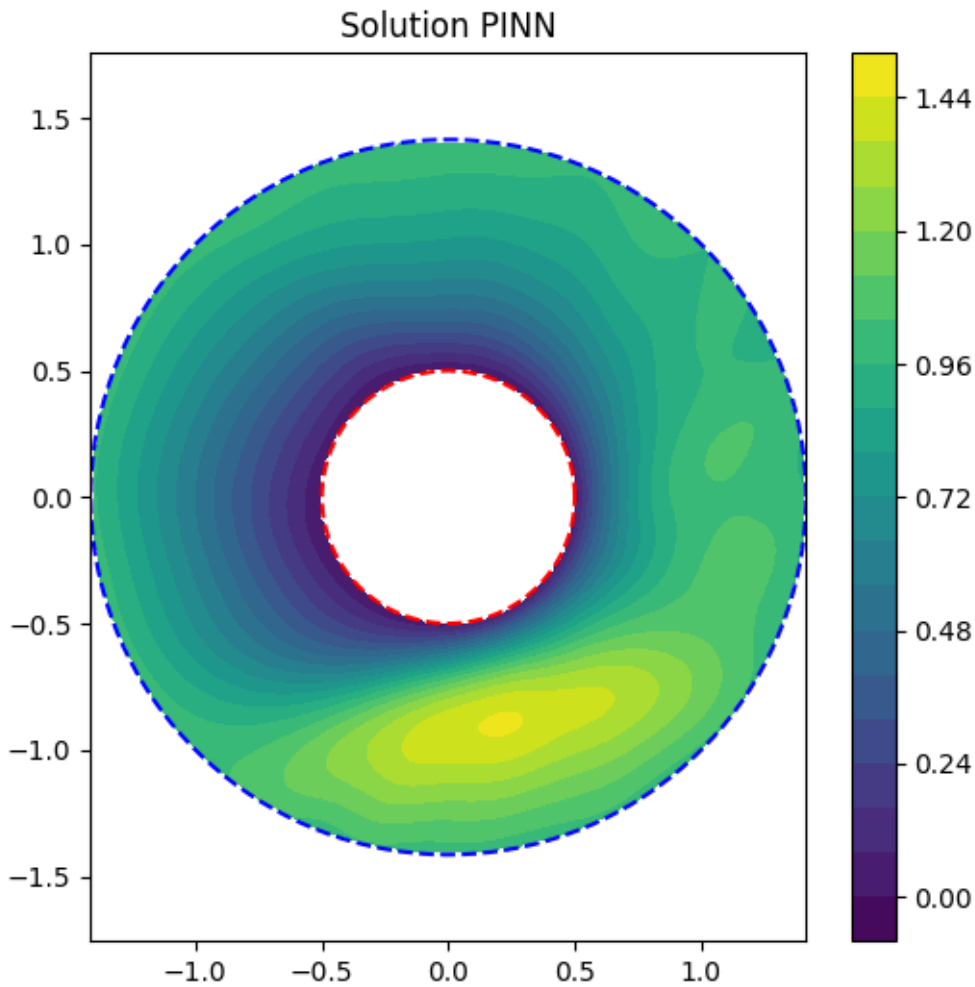
[12]:
```python
from src.models.pinn.trainers import train
from src.models.pinn.network import PINN
import torch
```

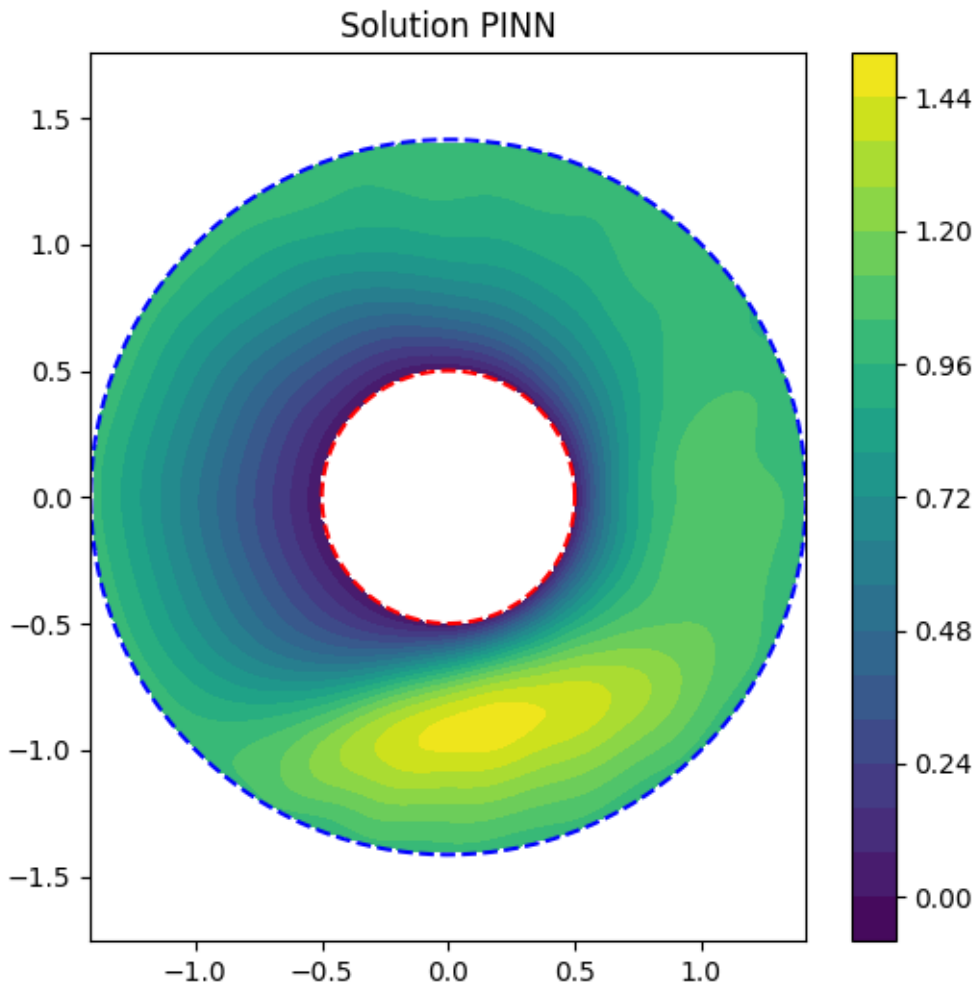
[13]:
```python
module = PINN(n_layers = 2)
lam_ = [10 * 2**k for k in range (10)]
for lam in lam_:
    train(module, params, f_1, lam = lam)
    # Grille d'évaluation
    N_plot = 200
    x = np.linspace(-params.R, params.R, N_plot)
    y = np.linspace(-params.R, params.R, N_plot)
    XX, YY = np.meshgrid(x, y)

    # Évaluation du modèle sur la grille
    xy_grid = np.stack([XX.ravel(), YY.ravel()], axis=1)
    with torch.no_grad():
        U = module(torch.tensor(xy_grid, dtype=torch.float32)).numpy().
 ↪reshape(N_plot, N_plot)

    # Masque anneau
    rad = np.sqrt(XX**2 + YY**2)
    mask = (rad > params.r) & (rad < params.R)
    U[~mask] = np.nan

    # Plot
    theta = np.linspace(0, 2*np.pi, 300)
    plt.figure(figsize=(6, 6))
    plt.contourf(XX, YY, U, 20, cmap='viridis')
    plt.colorbar()
    plt.plot(params.r*np.cos(theta), params.r*np.sin(theta), 'r--', lw=1.5)
    plt.plot(params.R*np.cos(theta), params.R*np.sin(theta), 'b--', lw=1.5)
    plt.axis('equal')
    plt.title('Solution PINN')
```

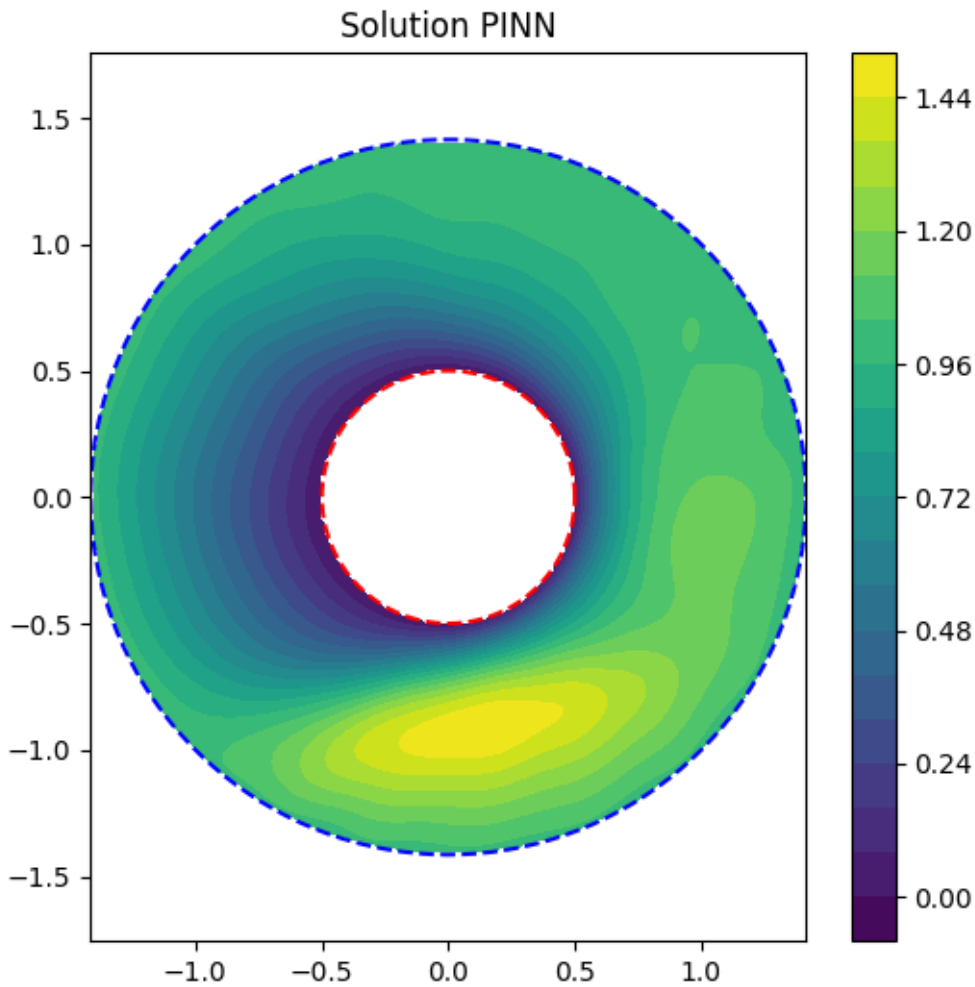
```
plt.show()
```

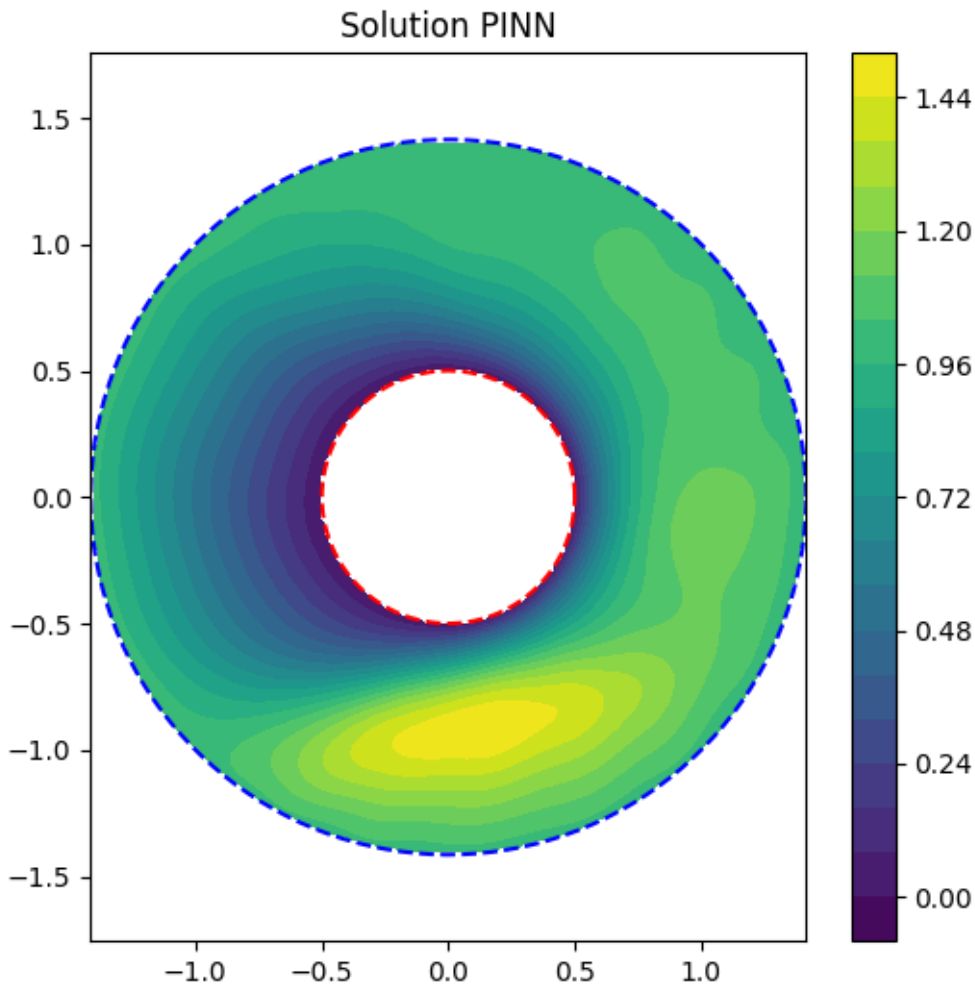

Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.08
0.90
0.72
0.54
0.36
0.18
0.00

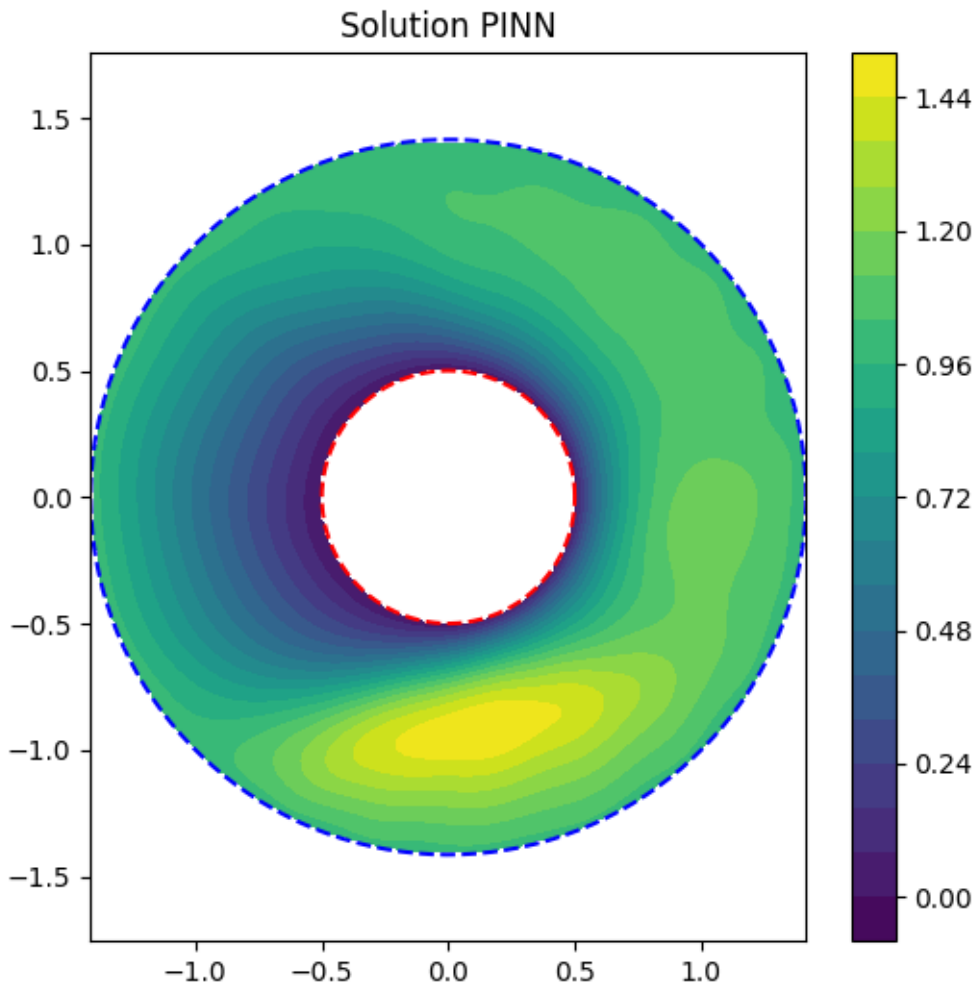

Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.28
1.12
0.96
0.80
0.64
0.48
0.32
0.16
0.00

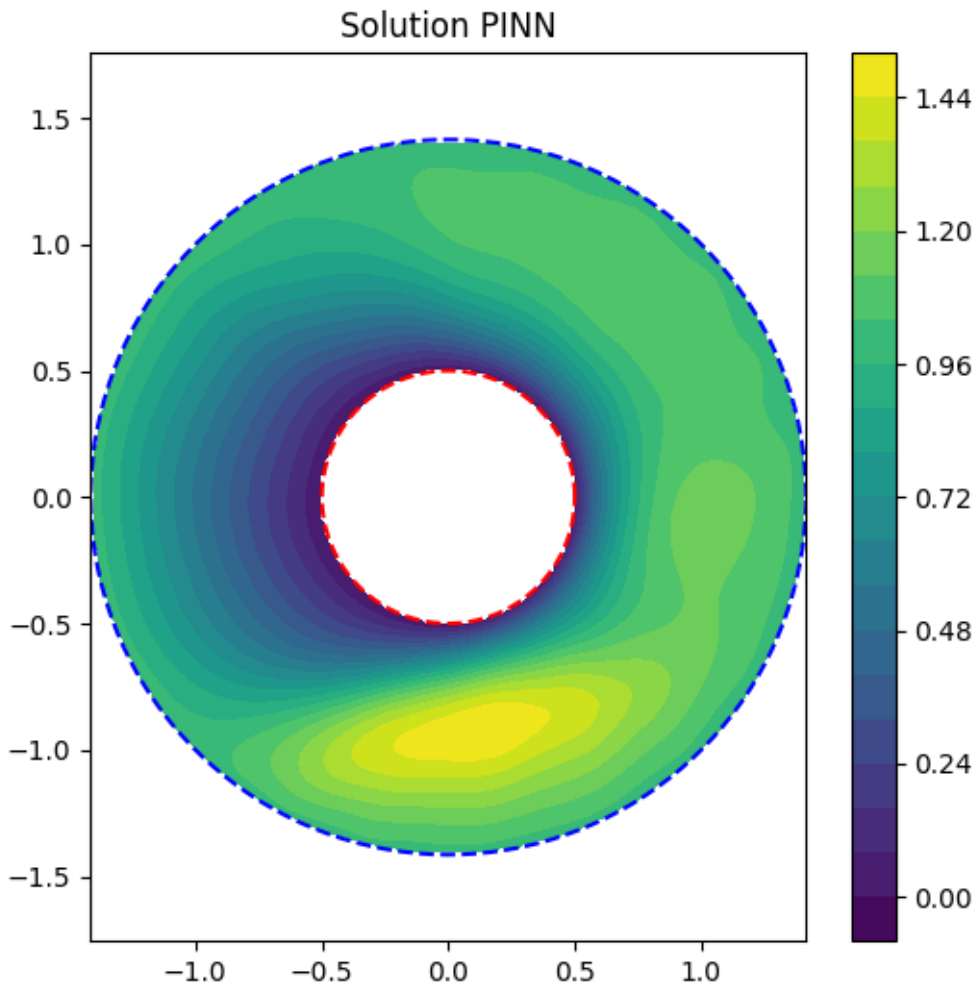

Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.44
1.20
0.96
0.72
0.48
0.24
0.00

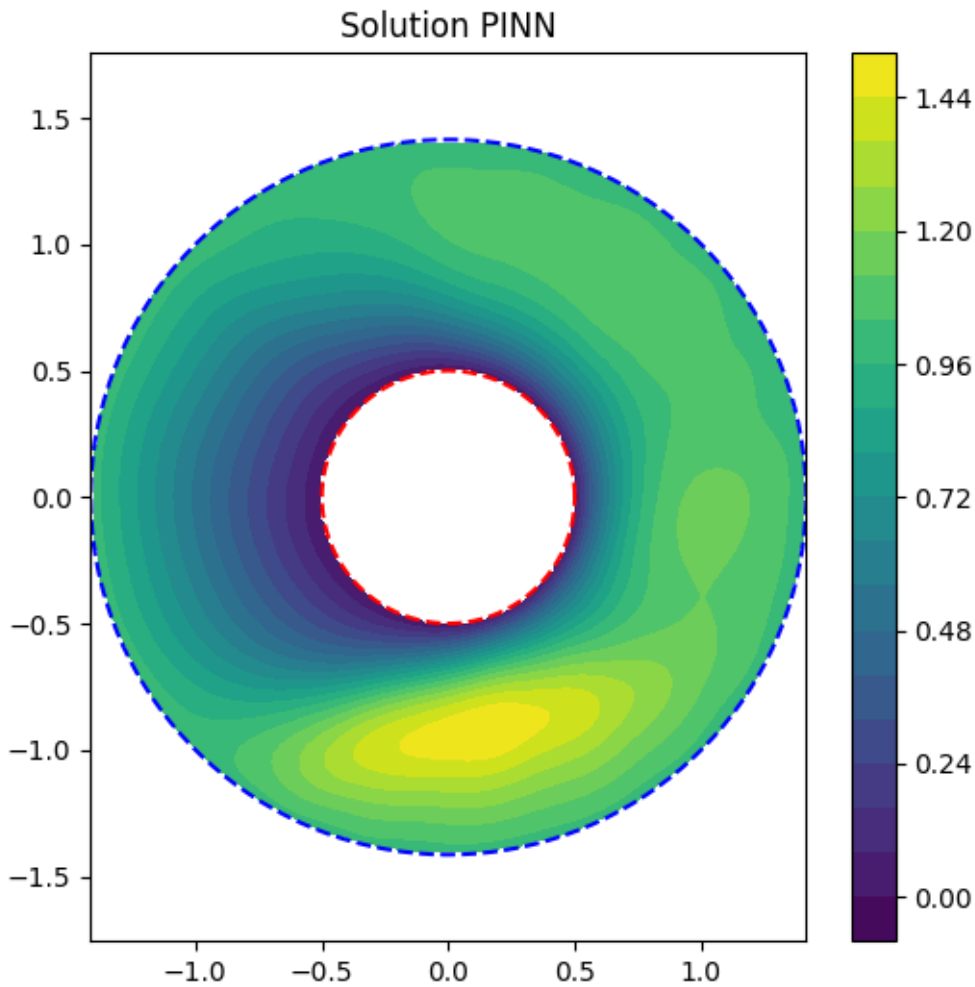

Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.44
1.20
0.96
0.72
0.48
0.24
0.00

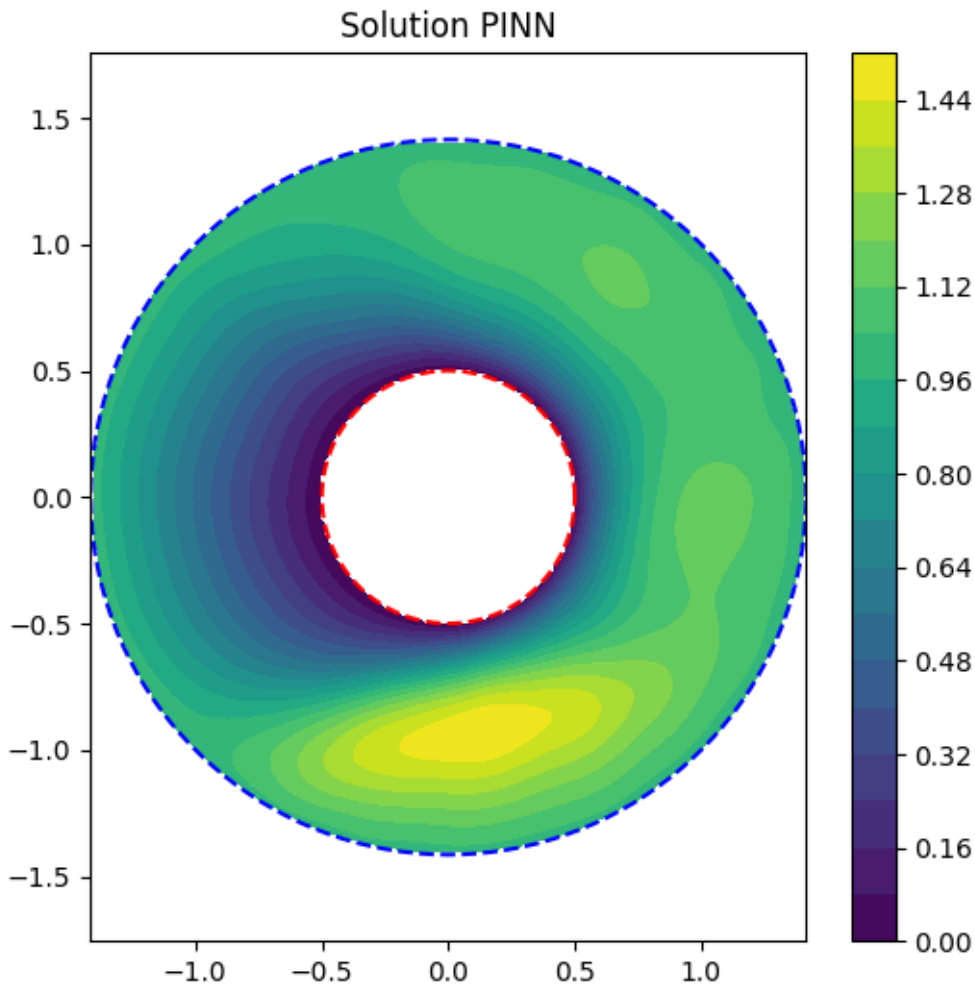

Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.44
1.20
0.96
0.72
0.48
0.24
0.00


Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.44
1.20
0.96
0.72
0.48
0.24
0.00


Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.44
1.20
0.96
0.72
0.48
0.24
0.00

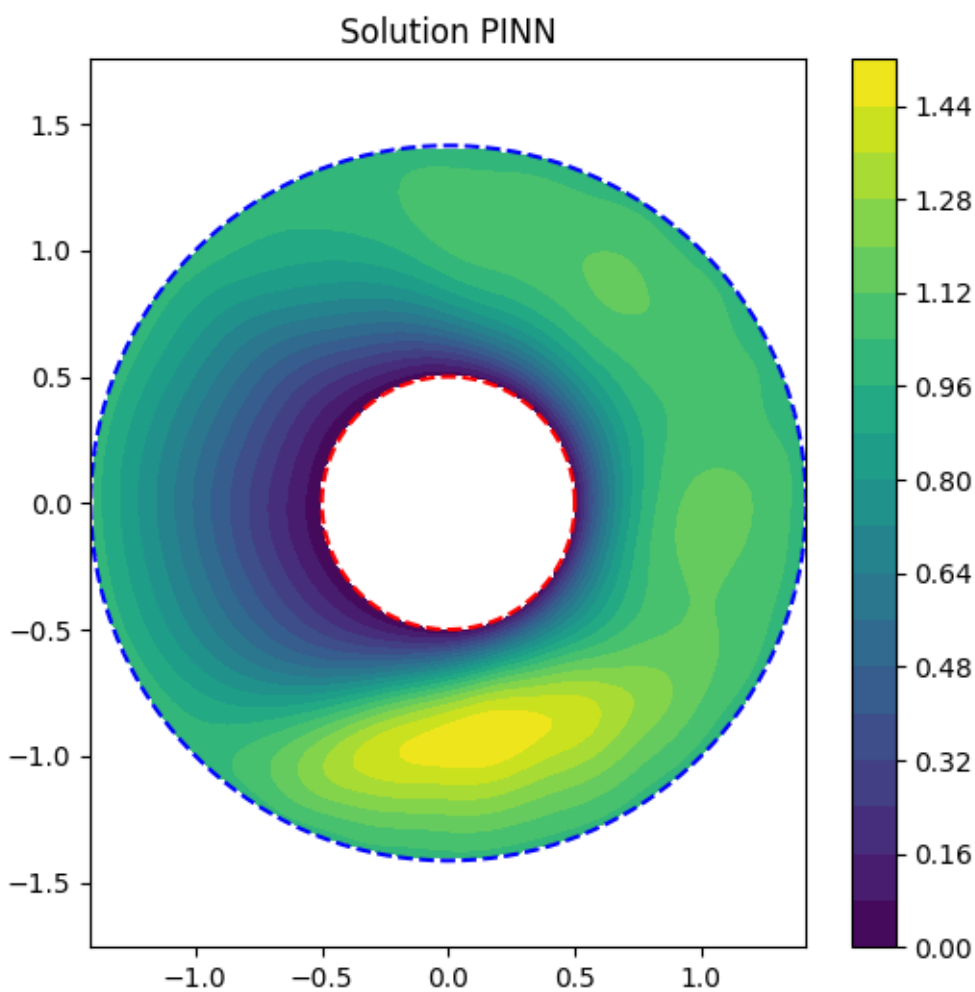

Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.44
1.20
0.96
0.72
0.48
0.24
0.00


Solution PINN
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.44
1.20
0.96
0.72
0.48
0.24
0.00



[14]:
```
# Grille d'évaluation
N_plot = 200
x = np.linspace(-params.R, params.R, N_plot)
y = np.linspace(-params.R, params.R, N_plot)
XX, YY = np.meshgrid(x, y)

# Évaluation du modèle sur la grille
xy_grid = np.stack([XX.ravel(), YY.ravel()], axis=1)
with torch.no_grad():
    U = module(torch.tensor(xy_grid, dtype=torch.float32)).numpy().
 ↪reshape(N_plot, N_plot)

# Masque anneau
rad = np.sqrt(XX**2 + YY**2)
mask = (rad > params.r) & (rad < params.R)
U[~mask] = np.nan
```

```
# Plot
theta = np.linspace(0, 2*np.pi, 300)
plt.figure(figsize=(6, 6))
plt.contourf(XX, YY, U, 20, cmap='viridis')
plt.colorbar()
plt.plot(params.r*np.cos(theta), params.r*np.sin(theta), 'r--', lw=1.5)
plt.plot(params.R*np.cos(theta), params.R*np.sin(theta), 'b--', lw=1.5)
plt.axis('equal')
plt.title('Solution PINN')
plt.show()
```



[ ]:

```
```